SWU 003 — System Design & Governance Document Sections

Project: Optimizing Resource Allocation in Disaster Relief Operations Through Web-Based Donation System
Client: Rise Above Foundation Cebu | **Institution:** Southwestern University PHINMA
Team Members: Julie Anne Utrera, Aubrie Marie Dual, Sheena Marie Dinaguit, James Lloyd Isidro
Sections: 3.5 Database Metadata, 3.6 Environments, 3.7 Service Accounts, and 4.2.2 – 4.2.7 Governance Standards

3.5. Database Metadata

The system utilizes a relational database management system hosted on **TiDB Cloud** (Serverless MySQL 8.0 compatible engine) operating with `utf8mb4` character set and `utf8mb4_general_ci` collation. The complete schema comprises twenty-seven (27) normalized tables structured across user role-based access control, donation lifecycles, relief inventory tracking, kit repacking operations, calamity distribution logistics, volunteer shift scheduling, and cryptographically verifiable digital certificates.

Database Metadata Link & Separate File Specification:
• GitHub Repository Database Schema Link:
https://github.com/jamis-ops/Donation-Management-System/blob/main/database/donation_system.sql

• Complete Data Dictionary & Metadata Documentation Link:
https://github.com/jamis-ops/Donation-Management-System/blob/main/database/DATABASE_METADATA.md

Summary of Database Tables & Entity Scope

Module	Table Name	Primary Key	Description & Purpose
Authentication	`roles`	id (INT)	Defines system privilege tiers (Admin, Staff, Donor, Volunteer, Beneficiary).
Authentication	`users`	id (BIGINT)	Central user identities, bcrypt-hashed passwords, profile status, and audit logs.
Authentication	`password_resets`	id (BIGINT)	Time-limited 6-digit recovery codes, temp session tokens, and brute-force throttling counters.
Donations	`donors`	id (BIGINT)	Individual and corporate donor profiles, unique donor codes, and organization links.
Donations	`donations`	id (BIGINT)	Monetary and in-kind donation ledgers, public tracking codes, payment proofs, and statuses.
Donations	`donation_updates`	id (BIGINT)	Real-time timeline audit trail (Received, Inspected, Allocated, Dispatched).

Module	Table Name	Primary Key	Description & Purpose
Donations	`donation_procurements`	id (BIGINT)	Direct allocations linking monetary donation funds to specific relief item purchases.
Beneficiaries	`beneficiaries`	id (BIGINT)	Barangay community units, evacuation centers, family profiles, and vulnerability scores.
Beneficiaries	`need_types`	id (INT)	Configurable lookup catalog of assistance categories.
Beneficiaries	`barangay_types`	id (INT)	Categorization of barangay jurisdictions across Cebu province.
Beneficiaries	`assistance_requests`	id (BIGINT)	Formal calamity assistance requests with urgency ratings and workflow tracking.
Inventory	`inventory_items`	id (BIGINT)	Warehouse relief stock, quantities on hand, reorder thresholds, and expiration dates.
Inventory	`repacking_jobs`	id (BIGINT)	Work orders bundling individual stock items into standardized family food packs.
Distribution	`distributions`	id (BIGINT)	Disaster relief missions, barangay venues, dispatch logs, and completion verification.
Distribution	`allocations`	id (BIGINT)	Linkage between assembled relief packs and specific distribution batches.
Distribution	`distribution_proofs`	id (BIGINT)	Field-uploaded photo evidence, delivery manifests, and beneficiary sign-offs.
Volunteers	`volunteers`	id (BIGINT)	Volunteer roster, logged duty hours, skills, and Google Gemini AI CV summaries.
Volunteers	`volunteer_schedule`	id (BIGINT)	Duty shifts, calendar schedules, and operational assignments.
Volunteers	`tasks`	id (BIGINT)	Operational task dispatch board with statuses and assigned volunteer IDs.
Volunteers	`task_types`	id (INT)	Lookup categorization of relief duties.
Certificates	`certificates`	id (BIGINT)	Verifiable digital certificates with unique hash codes and public QR validation.
Public Content	`programs`	id (BIGINT)	Community outreach initiatives and disaster preparation programs.

Module	Table Name	Primary Key	Description & Purpose
Public Content	`announcements`	id (BIGINT)	Public bulletins, relief operation updates, and disaster advisories.
Public Content	`partners`	id (BIGINT)	Collaborating NGOs, corporate sponsors, and government LGU partners.
Public Content	`success_stories`	id (BIGINT)	Published community impact stories and relief distribution transparency logs.
System	`notifications`	id (BIGINT)	Transactional system alerts delivered to user dashboards.
System	`admin_section_views`	id (BIGINT)	Tracks administrative read/audit states across sub-sections.

3.6. Environments

To guarantee system availability, protect live operational data, and ensure robust staging before deployment, the platform is separated into three tiers: **Production**, **User Acceptance Testing (UAT)**, and **Test (Development)**.

For the System (Frontend Web Application):

Production	UAT	Test
Production Frontend (Vercel CDN) https://donation-system-frontend.vercel.app	**UAT / Staging (Vercel)** https://donation-system-uat.vercel.app	**Local Development Server** `http://localhost:5173`

For the API (Backend RESTful Web Services):

Production	UAT	Test
Production REST API (Render Docker) https://donation-system-api.onrender.com/api	**UAT Staging API (Render)** https://donation-system-api-uat.onrender.com/api	**Local PHP Server CLI** `http://localhost:8000/api`

For the Database (MySQL 8.0 / TiDB Cloud Serverless):

Production	UAT	Test
TiDB Cloud Production Cluster `gateway01.ap-southeast-1.prod.aws.tidbcloud.com:4000` Database: `donation_system` (TLS/SSL Encrypted)	**TiDB Cloud UAT Staging Branch** `gateway01.ap-southeast-1.prod.aws.tidbcloud.com:4000` Database: `donation_system_uat`	**Local Test Database** `127.0.0.1:3306` / `local_test.sqlite` Database: `donation_system_test`

3.7. Service Accounts

Service accounts handle automated background operations, transactional email broadcasts (Brevo API), AI skill recommendation pipelines (Google Gemini AI), automated database schema migrations, and elevated administrative tasks.

For the System:

Production	UAT	Test
Primary Super Admin: `admin@riseabovefoundation.org`	**UAT Lead Administrator:** `uat-admin@riseabovefoundation.org`	**Local Dev Superuser:** `test-admin@localhost.dev`

Production	UAT	Test
Automated Notification Mailer: `noreply@riseabovefoundation.org` **CI/CD & Cloud Infrastructure:** `devops@riseabovefoundation.org`	**UAT Notification Mailer:** `uat-mailer@riseabovefoundation.org` **UAT QA Coordinator:** `uat-tester@riseabovefoundation.org`	**Mock Mailer Service:** `mailhog@localhost.dev` **Capstone Lead Developer:** `jamesisidro0808@gmail.com`

4.2. Governance Standards

4.2.2. Code Naming Convention

Purpose: Establishing strict code naming conventions ensures maintainability, team velocity, readability, and consistency across a full-stack polyglot environment comprising a React 18/19 SPA frontend, a PHP 8.2 backend, and a relational MySQL/TiDB database.

Specific Casing Styles and Scopes:

- **PascalCase:** Exclusively applied to React UI Components (e.g., `StatsSection.jsx`, `PartnersSection.jsx`, `AdminLayout.jsx`), React Context Providers (`AuthContext.jsx`, `ThemeContext.jsx`), and PHP Object-Oriented Classes/Exceptions.
- **camelCase:** Applied to JavaScript variables, React custom hooks (e.g., `useApiList`, `usePagination`, `useAuth`), component props, JavaScript helper functions (e.g., `fetchDonationStats()`, `handleVerify()`), and JSON API payload attributes.
- **snake_case:** Applied to all database tables and column names (e.g., `inventory_items`, `tracking_code`, `proof_file`, `created_at`) and backend PHP variables, database query parameters, and procedural helper scripts (e.g., `$db_host`, `get_db_connection()`).
- **UPPER_SNAKE_CASE:** Strictly reserved for global constants, configuration definitions, and environment variables (e.g., `DB_HOST`, `DB_PASSWORD`, `JWT_SECRET`, `VITE_API_URL`, `MAX_UPLOAD_SIZE_MB`).
- **kebab-case:** Applied to CSS class names (e.g., `.stat-card`, `.btn-primary`, `.modal-backdrop`), web route URLs (e.g., `/admin/inventory-management`, `/portals/donor/certificates`), and REST API file/resource endpoints.

Prefixes and Suffixes:

- **Booleans:** Must begin with affirmative verbs such as `is`, `has`, `can`, or `should` (e.g., `isLoading`, `isVerified`, `hasPermission`, `canDistribute`).
- **React Custom Hooks:** Must invariably start with `use` (e.g., `useApiList`, `usePagination`, `useAuth`).
- **Event Handlers:** Must use `handle` for definition (e.g., `handleDonationSubmit`) and `on` for passed callback props (e.g., `onSubmit`, `onStatusChange`).
- **Foreign Keys:** Must always be suffixed with `_id` matching the singular referenced table (e.g., `user_id`, `donor_id`, `role_id`).

Rules for Abbreviations: Cryptic abbreviations are prohibited. Standard industry abbreviations are permitted: `id`, `db`, `api`, `url`, `cv`, `auth`, `lgu`. Business models must use full descriptive terms (use `beneficiary` rather than `ben`, `repacking_job` rather than `rpk_j`).

Exceptions: Environment variables and constants are written in `ALL_CAPS`. Embedded acronyms in camelCase maintain consistent casing (e.g., `parseCsvFile`, `fetchApiData`).

4.2.3. Coding Standards

Overall Coding Philosophy: The team adheres to **DRY** (Don't Repeat Yourself) via centralized utility libraries and bootstrap handlers, **SOLID** design principles for modular API routes, **KISS** (Keep It Simple, Stupid) for

accessible logic, and **Defensive Programming** via parameterized PDO SQL queries, rigorous input sanitization, and type-casting.

Style Guides Referenced:

- **Frontend (JavaScript / React):** Airbnb JavaScript Style Guide and official React.dev guidelines.
- **Backend (PHP):** PSR-12 (Extended Coding Style) and PSR-4 (Autoloading Standards) defined by PHP-FIG.
- **Database (SQL):** SQL Style Guide (ISO/IEC 9075) with uppercase reserved keywords and lowercase identifiers.

Formatting Rules: 2-space soft indentation for JavaScript, JSX, and CSS; 4-space indentation for PHP backend code. Maximum line width is 100 characters. Semicolons are mandatory. 1TBS (One True Brace Style) is enforced across JavaScript.

Documentation: JSDoc and PHPDoc blocks are required on all shared utilities, custom hooks, and API endpoints indicating parameters, return types, and thrown exceptions. Inline comments clarify architectural rationale ('why') rather than obvious code logic ('what').

Error Handling Conventions:

API responses conform strictly to a standardized JSON response envelope:

```
{
  "success": true | false,
  "data": { ... } | null,
  "message": "Descriptive operational message",
  "error": "Standardized error code or null"
}
```

Explicit HTTP status codes must be returned (200, 201, 400, 401, 403, 404, 422, 500). Internal database errors are logged securely server-side without leaking raw credentials or stack traces to client browsers.

Enforcement Tools: ESLint and Prettier for frontend formatting; PHP_CodeSniffer (phpcs) for backend PSR-12 validation; Git pre-commit hooks configured via Husky to block non-conforming commits.

4.2.4. Environment Variables Standards

Purpose: Adhering to Twelve-Factor App methodology (Config III), all environment-specific configs, database secrets, and API credentials are completely externalized from code, ensuring zero leakage into git and seamless portability across dev, staging, and production.

Naming & Categorization:

- `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_SSL` — Database connectivity credentials.
- `BREVO_API_KEY`, `GEMINI_API_KEY`, `MAIL_FROM_EMAIL`, `MAIL_FROM_NAME` — External cloud services.
- `VITE_API_URL`, `VITE_APP_ENV`, `VITE_APP_VERSION` — Frontend client-accessible environment flags.
- `FRONTEND_URL`, `SESSION_LIFETIME`, `APP_SECRET` — CORS security and session lifetime tokens.

Storage Locations: Local development uses an uncommitted `.env` file. Staging and Production utilize Render Environment Variables Dashboard and Vercel Project Environment Variables. A sanitized template `.env.example` is committed with clear documentation.

Hardcoding Policy: Passwords, database endpoints, encryption keys, and private tokens must NEVER be hardcoded. Permitted hardcoded items are limited to non-sensitive defaults (e.g., default page limit of 10, static UI strings).

Security Practices: `.env*` files are permanently excluded in `.gitignore`. Production secrets are restricted to designated administrators and rotated on a 90-day cycle or immediately upon team member offboarding.

4.2.5. Folder Structure

Organizing Principle: The codebase adopts a Layered, Decoupled Client-Server Modular Architecture separating the frontend React SPA (`/Frontend`), the stateless PHP REST API (`/Backend`), and database migration assets (`/database`).

Folder Path	Layer / Scope	Purpose & Architecture
`/Backend/api/`	Backend API	PHP REST endpoints (donations, inventory, distributions, auth), session handling, and CORS middleware.
`/Backend/api/uploads/`	Storage	Secure upload directory for distribution proof photos, donation receipts, and volunteer CVs.
`/Frontend/src/pages/`	Frontend Views	Top-level public views (Home, AboutUs, FAQ, Donation Tracker, Login, Register).
`/Frontend/src/portals/`	Role Portals	Dedicated portal modules and dashboards (Admin, Donor, Volunteer, Beneficiary, Staff).
`/Frontend/src/components/`	UI Components	Modular UI widgets grouped into `/admin`, `/layout`, `/shared`, and `/home`.
`/Frontend/src/context/`	State Management	Global React Context state providers (`AuthContext.jsx`, `ThemeContext.jsx`).

Folder Path	Layer / Scope	Purpose & Architecture
`/Frontend/src/hooks/`	Custom Logic	Reusable custom React hooks (`useApiList.js` , `usePagination.js`).
`/Frontend/src/styles/`	Styling	CSS stylesheets, CSS variable design tokens, and role-specific theme styling.
`/database/`	Persistence	Master SQL schema files (`donation_system.sql`), seed fixtures, and `DATABASE_METADATA.md` .
`/mail-service/`	Microservice	Express/Node.js background mail notification microservice for queueing transactional notices.

Nested Rules: Reusable components shared across two or more portals must reside in `/components/shared/` . Folder names follow kebab-case for backend microservices and PascalCase for React component folders.

4.2.6. Taxonomy/Metadata Standards

Purpose: Establishing taxonomy standards ensures unified classifications, predictable API contracts, and unambiguous semantic versioning across all system layers.

- **Database Schema Metadata:** Tables are pluralized snake_case nouns (e.g., `assistance_requests`). Primary keys are consistently `id BIGINT UNSIGNED AUTO_INCREMENT` . Foreign keys strictly follow `singular_parent_table_id` . Mandatory audit columns `created_at` and `updated_at` are present on all transaction tables.
- **API Metadata:** Responses include standardized HTTP headers (`Content-Type: application/json` , `X-API-Version: 1.0.0`). Paginated responses provide pagination metadata envelopes (`total_records` , `current_page` , `per_page` , `total_pages`).
- **Code Metadata:** PHPDoc and JSDoc tags (`@param` , `@returns` , `@throws` , `@deprecated`) are required on all shared functions.
- **Version Control Tagging:** Semantic Versioning (SemVer 2.0.0) format (`vMAJOR.MINOR.PATCH`) is enforced (e.g., `v1.0.0` Initial Capstone Release). Commits adhere to Conventional Commits (`feat:` , `fix:` , `docs:` , `refactor:` , `chore:`).
- **Feature Flags:** Runtime flags (e.g., `FEATURE_AI_VOLUNTEER_MATCHING=true` , `MAINTENANCE_MODE=false`) control functionality without code rebuilds.

4.2.7. Deployment Standards

Deployment Environments: 1) Development (Local machine `localhost:5173` / `localhost:8000`); 2) UAT (Staging on Vercel and Render) for validation by Rise Above Foundation Cebu staff and SWU faculty panelists; 3) Production (Vercel Global CDN + Render Docker + TiDB Cloud Serverless) for live humanitarian relief operations.

CI/CD Deployment Process:

- **1. Code Commit:** Developer pushes code to GitHub repository branches.
- **2. Automated Trigger:** GitHub webhooks trigger Render and Vercel build pipelines.

- **3. Build & Test:** Vercel compiles and bundles the React SPA; Render compiles the Docker container with PHP 8.2 Apache.
- **4. Health Check Verification:** Render checks `/api/health.php` for zero-downtime container readiness before routing live traffic.
- **5. Database Migrations:** Schema changes are applied via transaction-safe migration scripts against TiDB Cloud.

Branching Strategy: Structured GitFlow model:

- `feature/*`: Feature branches branched from `develop`.
- `develop`: Continuous integration branch automatically deployed to **UAT / Staging**.
- `main`: Protected production branch. Direct pushes are disabled. Deploys to **Production**.
- `hotfix/*`: Critical production patches branched from `main` and back-merged to `develop`.

Approval Requirements: Merges to `main` require a formal Pull Request (PR) with at least two peer reviews and green status from automated build checks, authorized by the Lead Developer.

Rollback Procedures:

- **Frontend Rollback (Vercel):** Vercel Dashboard -> Deployments -> Select prior stable build -> Click "Rollback to this Deployment" (executes in under 15 seconds).
- **Backend Rollback (Render):** Render Dashboard -> Service Activity -> Select prior successful Docker image -> Trigger redeploy.
- **Database Rollback (TiDB Cloud):** Point-In-Time Recovery (PITR) snapshots allow restoring the database to any exact second prior to deployment.